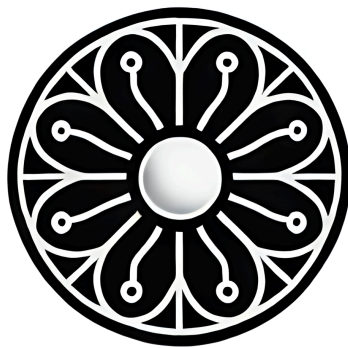


FACULTY OF ENGINEERING

CAIRO UNIVERSITY

DEPARTMENT OF COMPUTER ENGINEERING



PEARL

A Graduation Project Report Submitted
to
Faculty of Engineering, Cairo University
in Partial Fulfillment of the requirements of the degree
of
Bachelor of Science in Computer Engineering.

Presented by

Youssef Mohamed Rabie

Supervised by

DR. Mostafa Mahmoud

July, 2025

All rights reserved. This report may not be reproduced in whole or in part, by photocopying or other means, without the permission of the authors/department.

Abstract

Reinforcement Learning (RL) is a powerful framework in which agents learn to make decisions by interacting with an environment to maximize cumulative rewards. Traditionally, the effectiveness of an RL agent is judged by scalar reward signals accumulated during training or evaluation episodes. While useful in many settings, such metrics often fall short in capturing *why* an agent behaves in a particular way, or whether its learned strategy aligns with intended behaviors—especially in complex, safety-critical, or poorly defined environments. Moreover, two agents with similar total rewards may have learned drastically different policies, some of which may be brittle, unsafe, or unintuitive.

This project explores the use of explainability tools as an alternative—and potentially more insightful—method for evaluating and comparing reinforcement learning agents. By applying interpretability techniques such as SHAP (SHapley Additive exPlanations), LIME (Local Interpretable Model-agnostic Explanations), and saliency mapping, we aim to visualize and quantify how agents make decisions, what features they rely on, and how these internal justifications evolve over training. We investigate whether these explanations can serve not only as debugging tools, but also as evaluation criteria to detect reward hacking, policy overfitting, or divergent learning strategies that traditional metrics may miss.

Experiments are conducted in several benchmark environments, including both discrete control tasks and pixel-based domains. We compare agents that achieve similar rewards but differ in their internal decision logic, using explainability outputs to highlight key differences. The results show that interpretable evaluation can expose subtle yet critical distinctions in agent behavior, providing a complementary lens through which to assess RL performance. This work concludes by discussing the potential of explainability to act as a diagnostic or benchmarking tool, especially in domains where raw rewards are insufficient or misleading.

The technical implementation of the system was built using Python for the reinforcement learning framework and experiment management, C++ for building a graphical user interface (GUI) that displays agent behavior and decisions, and NS-3 for simulating realistic network environments where RL agents interact.

This project was proudly sponsored by **Ericsson**, whose support enabled access to helpful resources and feedback, especially in the context of learning about RL explainability and applying RL techniques to communication networks.

Abstract.....	1
Chapter 1: Introduction.....	4
1.1 Motivation and Justification.....	4
1.2 Project Objectives and Problem Definition.....	4
1.3 Project Outcomes.....	4
Chapter 2: Team Leadership Contributions.....	6
2.1 Team Coordination and Communication.....	6
2.2 Project Planning and Resource Management.....	7
2.3 Task Assignment and Deadline Tracking.....	7
2.4 Summary.....	7
Chapter 3: Knowledge Acquisition and Technical Preparation.....	8
3.1 Investigated Papers:.....	8
3.1.1 “BEERL: Both Ends Explanations for Reinforcement Learning”.....	8
3.1.1.1 Overview:.....	8
3.1.1.2 Fine-Grained Explanations and Key Contributions of BEERL.....	8
3.1.1.3 How BEERL Differs from Other XRL Methods.....	9
3.1.1.4 Relevance to Our Work:.....	10
3.1.2 “Toward Interpretable Deep Reinforcement Learning with Linear Model U-Trees”. ..	11
3.1.2.1 Overview.....	11
3.1.2.2 Mimic Learning Framework for DRL.....	11
3.1.2.3 Mimic Learning Framework for DRL.....	11
3.1.2.4 Training Procedure.....	12
3.1.2.5 Contributions.....	12
3.1.2.6 Relevance to Our Project.....	13
3.2 Self-Learning Materials.....	13
3.2.1 “Deep Reinforcement Learning for Atari Games Python Tutorial” AI Plays Space Invaders by Nicholas Renott.....	13
3.2.2 “DeepMind x UCL Reinforcement Learning Course” by David Silver.....	13
3.2.3 “Reinforcement Learning Course – Full Machine Learning Tutorial” by freeCodeCamp.....	14
3.2.4 “Explainable AI” by DeepFindr.....	14
3.2.5 “CMP 4010: Machine Intelligence” by Dr. Yahia Zakaria.....	14
3.2.6 “CMP 4040: Machine Learning” by Dr. Yahia Zakaria.....	14
Chapter 4: Contributions.....	15
4.1 Initial Experiment: Interpreting RL Agent Behavior in Space Invaders.....	15
My Contributions:.....	15
Results and Insights:.....	15
4.2 Implementing Linear Model U-Trees (LMUT).....	17
4.2.1 Core Idea.....	17
4.2.2 Implementation Highlights.....	17

4.2.2.1 Data Collection Engine.....	18
4.2.2.2 Decision Tree Construction.....	18
4.2.2.3 Per-Leaf Linear Model Training.....	19
4.2.2.4 Explanation Generation.....	19
4.2.2.5 Integration with Feature Masking.....	19
4.2.2.6 Sample Code Snippet – LMUT Training & Explanation.....	20
4.2.3 Practical Value.....	20
4.2.4 Reflections and Future Enhancements.....	21
4.2.5 Results.....	21
4.3 Integration of the Linear Model U-Trees (LMUT) Method into the Library and GUI.....	22
4.4 Deployment of the PEARL Library to PyPI.....	23
4.4.1 Overview.....	23
4.4.2 Installation.....	23
4.4.3 Packaging and Build Process.....	23
4.4.4 Deployment Script Snippet:.....	24
4.4.5 Testing the Package.....	24
4.4.6 Summary.....	24
References.....	25

Chapter 1: Introduction

1.1 Motivation and Justification

Reinforcement learning has shown remarkable success across various domains, from robotics and control systems to games and communication networks. However, evaluating the performance of RL agents is still predominantly based on scalar reward signals—metrics that may not always reflect the quality, stability, or safety of the learned policy. Two agents may receive the same total reward but behave very differently under the hood. In applications where explainability and reliability are critical—such as autonomous systems or network optimization—this limitation becomes problematic. This project is motivated by the growing need for more interpretable and robust evaluation frameworks that go beyond rewards and instead focus on understanding the agent’s internal decision-making logic.

1.2 Project Objectives and Problem Definition

The primary objective of this project is to develop and assess a framework that uses explainability tools to compare RL agents based on their internal behavior rather than external reward signals. We aim to investigate whether interpretability techniques can be used not only for debugging and model introspection but also as a core evaluation metric. The problem this thesis addresses is: *How can we use explainability to determine which of two RL agents is better, particularly when their reward signals are similar or misleading?* To this end, the project integrates SHAP, LIME, and visual saliency methods to explain and compare policy behaviors across agents and environments.

1.3 Project Outcomes

The project produced a working platform that combines reinforcement learning with explainability modules. Agents were trained in a variety of environments, and their decisions were interpreted using multiple techniques. The resulting insights helped differentiate between agents with similar scores but divergent strategies. The major outcomes include:

- A comparative analysis of reward-based and explanation-based evaluation.
- Visual dashboards showing agent attention and decision factors.
- Practical recommendations for using explainability in RL benchmarking.
- A working GUI built in C++ for interactive analysis.

Integration with NS-3 for network-based RL scenarios.

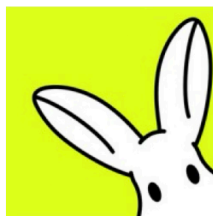
value of empathy, planning, and clear communication — qualities I believe will serve me far beyond this project.

Chapter 2: Team Leadership Contributions

At the beginning of the first semester, my team selected me to serve as the **Team Leader**, a role that came with a wide array of responsibilities and expectations. By the grace of Allah, the Most Gracious and Most Merciful, I was able to fulfill this role and contribute meaningfully to the project's direction and execution. In this section, I will summarize the most impactful aspects of my leadership.

2.1 Team Coordination and Communication

- **Team CV Compilation:** One of my first tasks as a team leader was to guide the team in creating a **comprehensive professional CV** that combined each member's information, achievements, technical experiences, and skills. The goal was to present our team as a unified, competent, and capable group to potential sponsors and collaborators.
- **Corporate Outreach:** I personally contacted over **20 companies** in search of sponsorship and support. As a result of these efforts, we received **six sponsorship offers**. We later held several meetings with our academic supervisor and among the team to evaluate the opportunities. After a thorough discussion, we made our final decision to proceed with **Ericsson** and to continue developing the **Pearl idea** — a decision based on both the technical value and real-world applicability of the project.



- **Supervisor Communication:** I was responsible for organizing and scheduling meetings with our academic supervisor. I also served as the main point of contact, drafting and sending all formal messages on behalf of the team.
- **Internal Meetings and Monitoring:** I organized weekly team meetings, set the agendas, and ensured everyone was on track. This helped maintain momentum and accountability across the entire semester.

2.2 Project Planning and Resource Management

- **Initial Brainstorming and Idea Selection:** I facilitated brainstorming sessions to evaluate each team member's skills and interests. This process guided us in selecting a meaningful and realistic project idea that aligned with our collective strengths.
- **Resource Organization:** To ensure seamless collaboration, I created a centralized **Google Drive folder** that housed all essential resources, including datasets, research papers, shared documents, and progress logs.
- **Research and Dataset Curation:** I helped collect relevant **datasets, research papers, and open-source code** that could guide our implementation. This early preparation saved us significant time later during development.

2.3 Task Assignment and Deadline Tracking

To eliminate confusion and avoid the “what should I do now?” feeling, I built and maintained a **task-tracking spreadsheet**. This tool allowed us to assign each task to a responsible member with a clear **deadline and priority**, making our workflow more efficient and less stressful.

2.4 Summary

In addition to the points listed above, my role extended into various micro-management and soft leadership responsibilities — from resolving conflicts and boosting team morale to double-checking reports before submission. These leadership duties taught me the value of empathy, planning, and clear communication — qualities I believe will serve me far beyond this project.

Chapter 3: Knowledge Acquisition and Technical Preparation

3.1 Investigated Papers:

3.1.1 “BEERL: Both Ends Explanations for Reinforcement Learning”

Authors: Ahmad Terra, Rafia Inam, and Elena Fersman (Ericsson & KTH)

3.1.1.1 Overview:

This paper introduces BEERL (Both Ends Explanations for Reinforcement Learning), a novel method designed to provide fine-grained explanations for Deep Reinforcement Learning (DRL) agents, which are typically black-box models due to their reliance on neural networks. Existing Explainable RL (XRL) methods often focus on either input-side explanations (e.g., feature importance) or output-side explanations (e.g., reward decomposition). BEERL, however, connects explanations from both ends of the black-box model, leading to more detailed and comprehensive insights. The method exposes reward prioritization to the user, generating two levels of explanation and enabling reconfigurations of the RL agent when undesirable behaviors are observed. BEERL further quantifies explanations through a 'focus value' that considers all reward components and measures the fulfillment of desired properties. The effectiveness of BEERL is demonstrated through its application to a remote electrical telecom-antenna-tilt use case and two OpenAI Gym environments (Lunar Lander and Cartpole), revealing fine-grained explanations of input features' contributions to specific rewards and identifying biases in reward components that can be mitigated by adjusting reward weights

3.1.1.2 Fine-Grained Explanations and Key Contributions of BEERL

The core strength of the BEERL framework lies in its ability to produce **fine-grained explanations** that link specific **input features** to individual **reward components**, rather than analyzing them in isolation. This dual perspective offers a deeper and more structured understanding of the decision-making process of reinforcement learning agents. BEERL achieves this through the following contributions:

1. **Reward Decomposition:** Splits the total reward into interpretable sub-rewards (e.g., signal quality, throughput, coverage) to better understand the agent's

priorities.

2. **Reward Prioritization Exposure:** Reveals how different reward weights influence decisions across two distinct levels: the **Q-function level** and the **RL level**.
3. **Bias Identification and Mitigation:** Detects biases in specific reward components and provides mechanisms to adjust them.
4. **Attributive and Contrastive Explanations:** Provides insights into *why* an action was chosen and *why not* others, enhancing decision transparency.
5. **Focus Value Metric:** Introduces a novel quantification of how well the agent's behavior aligns with user-defined expectations via a **relevance matrix**.
6. **Reconfigurable Agents:** Enables fine-tuning of agent behavior by adjusting reward priorities based on explanation feedback, improving trust and adaptability.

3.1.1.3 How BEERL Differs from Other XRL Methods

BEERL is different from other explainable reinforcement learning (XRL) methods in several important ways:

- **Correlation of Input and Output Explanations:** Unlike methods that focus solely on feature importance or reward decomposition, BEERL explicitly correlates input features with output reward components. This addresses a gap in existing literature where the contribution of input features to reward components, and vice-versa, was not adequately explored.
- **Granularity of Explanation:** While traditional methods might produce explanations based on the sum of input features (NI) and reward components (NC), BEERL can generate $NI \times NC$ explanations, offering a significantly more granular understanding of the agent's decision-making process.
- **Reward Weight Decoupling:** BEERL decouples reward weights and Q-function approximators, allowing each component to be treated equally and explicitly showing the weights. This contrasts with methods where all components are treated as having equal weights and only final reward component values are obtained.

- **Transparency vs. Distillation:** Instead of distilling the RL policy into a transparent model (as seen in some other XRL approaches), BEERL focuses on exposing the reward components and their proportions to make the second-to-last layer of the neural network transparent. This provides explanations at a finer level of granularity.
- **Quantification of Explanations:** BEERL introduces a focus value to quantify the desired properties set by the user, providing a measurable aspect to the quality of the explanation, which is often lacking in other XAI methods.

3.1.1.4 Relevance to Our Work:

This paper was foundational in the development of our **evaluation strategy**. It showed us how to:

- Interpret agent behavior from both the state (input) and reward (output) perspectives
- Use reward decomposition in a **modular way**, enabling us to track the performance of individual reward terms
- Implement **focus values and explanation matrices**, which we later adapted to evaluate PEARL agents' adherence to design expectations

Moreover, the BEERL method, being backed by Ericsson, directly connected with our **industrial collaboration**, offering real-world grounding and validation for our techniques.

3.1.2 “Toward Interpretable Deep Reinforcement Learning with Linear Model U-Trees”

Authors: Guiliang Liu, Oliver Schulte, Wang Zhu, and Qingcan Li

3.1.2.1 Overview

This paper addresses the challenge of **interpretability in Deep Reinforcement Learning (DRL)** models, especially those using neural networks to approximate Q-functions. Despite the impressive performance of DRL in various domains, the decision-making process remains opaque, which limits its adoption in real-world, safety-critical environments. To tackle this, the authors introduce a **mimic learning framework** and propose a novel structure called **Linear Model U-Trees (LMUTs)**.

The proposed framework allows for learning **interpretable approximations** of neural Q-functions by integrating the structure of decision trees with the expressiveness of linear models at the leaves. The LMUT approach supports **online learning**, making it practical for active interaction with an environment, a key requirement for real-time applications of RL.

3.1.2.2 Mimic Learning Framework for DRL

Unlike supervised learning, DRL involves learning from ongoing interaction with an environment rather than from fixed datasets. The paper presents two main training paradigms for mimic learning in DRL:

- **Experience Training:** Stores all transitions from a trained DRL agent and uses them offline to train an interpretable model. While practical, it may include suboptimal states and can be computationally expensive.
- **Active Play:** The mimic model observes the live interactions between the trained DRL agent and its environment, collecting data on-the-fly. This method avoids memory limitations and enables continuous online updates, making it better suited for real-time learning.

3.1.2.3 Mimic Learning Framework for DRL

Unlike supervised learning, DRL involves learning from ongoing interaction with an environment rather than from fixed datasets. The paper presents two main training paradigms for mimic learning in DRL:

- **Experience Training:** Stores all transitions from a trained DRL agent and uses them offline to train an interpretable model. While practical, it may include suboptimal states and can be computationally expensive.
- **Active Play:** The mimic model observes the live interactions between the trained DRL agent and its environment, collecting data on-the-fly. This method avoids memory limitations and enables continuous online updates, making it better suited for real-time learning.

3.1.2.4 Training Procedure

LMUT training is divided into two stages:

1. **Data Gathering Phase:** Transitions are collected and used to update statistics and build local models.
2. **Node Splitting Phase:** Leaf nodes are evaluated using **Stochastic Gradient Descent (SGD)**. If the fit is poor, the node is split using feature-based partitioning, and new linear models are trained in the resulting children.

This two-phase loop allows the model to **adapt dynamically** and maintain efficiency using **mini-batch learning**.

3.1.2.5 Contributions

The paper provides:

- The **first extension** of mimic learning techniques into DRL.
- A novel **online learning algorithm** that builds LMUTs incrementally.
- A demonstration of how to extract meaningful insights from DRL agents by interpreting the structure and parameters of the LMUT.

3.1.2.6 Relevance to Our Project

This paper directly inspired the **LMUT module I implemented** in our PEARL framework. It guided the:

- Structure of combining **decision trees with linear models**
- Implementation of **feature importance extraction**
- Design of the **data collection and model fitting process**

The practical insights from this paper helped me create an explainability layer that supports real-time agent evaluation while maintaining interpretability — a key requirement in our industry collaboration with Ericsson.

3.2 Self-Learning Materials

3.2.1 “Deep Reinforcement Learning for Atari Games Python Tutorial” | AI Plays Space Invaders by Nicholas Renott

This hands-on tutorial demonstrated how to build Deep Reinforcement Learning (DRL) agents to play **Space Invaders** using Python. It explained how to set up environments, define neural networks, and apply DRL techniques such as action selection and reward optimization. The practical focus helped me connect theoretical DRL ideas with real-world applications in high-dimensional spaces.

3.2.2 “DeepMind x UCL | Reinforcement Learning Course” by David Silver

This academic series offered a structured, rigorous introduction to reinforcement learning. It covered essential topics like **MDPs, Dynamic Programming, Monte Carlo methods, Temporal Difference learning**, and more advanced techniques such as policy gradients and actor-critic methods. It was key to understanding the mathematical and algorithmic foundations of RL.

3.2.3 “Reinforcement Learning Course – Full Machine Learning Tutorial” by freeCodeCamp

This course provided a wide-ranging overview of RL, including algorithms like **Q-learning, SARSA, and Deep Q-Networks (DQNs)**. Through practical examples and implementation exercises, it strengthened my grasp of how RL agents interact with environments and improve over time through trial and error.

3.2.4 “Explainable AI” by DeepFindr

This playlist introduced core XAI techniques including **SHAP, LIME**, saliency maps, and model-agnostic explanation strategies. It was crucial in shaping our explainability approach, especially in understanding how to interpret and visualize complex model decisions in reinforcement learning settings.

3.2.5 “CMP 4010: Machine Intelligence” by Dr. Yahia Zakaria

This university course covered the fundamentals of intelligent systems, including **search strategies, knowledge representation, and decision-making**. It helped establish my understanding of how agents perceive, reason, and act — foundational knowledge that supported my RL work.

3.2.6 “CMP 4040: Machine Learning” by Dr. Yahia Zakaria

Building on AI fundamentals, this course introduced practical ML topics such as **regression, classification, clustering, and neural networks**. It provided essential insights into model training, evaluation, and optimization — skills that were vital for agent development and performance tuning in our project.

Chapter 4: Contributions

4.1 Initial Experiment: Interpreting RL Agent Behavior in Space Invaders

As part of my early experimentation with explainability in reinforcement learning, I designed and conducted an initial study on the classic Atari game **Space Invaders**. The goal was to understand how different explainability methods behave when applied to a trained RL agent in a visual, high-dimensional environment.

My Contributions:

- Selected and configured the **Space Invaders** environment from the OpenAI Gym suite.
- Trained two reinforcement learning agents using **Proximal Policy Optimization (PPO)** and **Deep Q-Networks (DQN)**.
- Integrated post-hoc explainability methods, specifically **SHAP** and **LIME**, to interpret the decision-making process of the DQN agent.
- Analyzed and visualized the model's behavior in response to different game states.

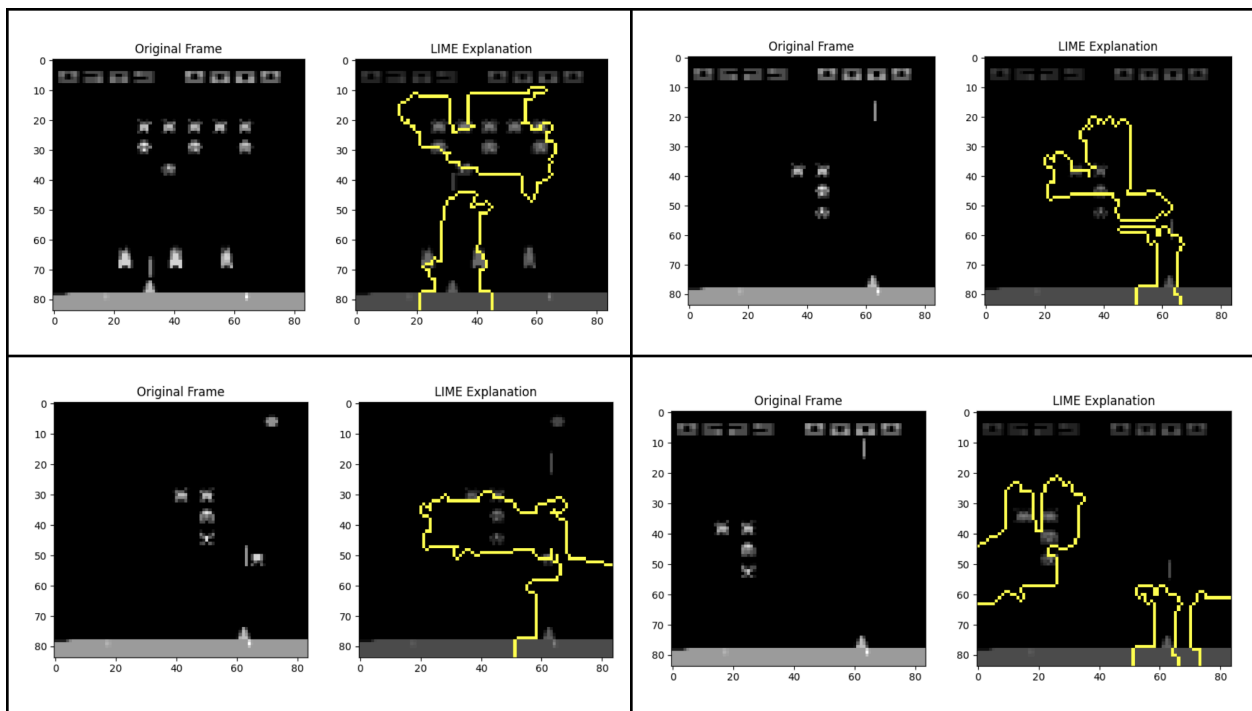
Results and Insights:

Using **LIME**, I observed that the agent's decisions were strongly influenced by two key factors:

- The **player's location** on the screen
- The **positions of the enemies** in the frame

This confirmed that the agent had learned to focus on relevant features of the environment. However, some explanation outputs were ambiguous or inconsistent, suggesting that the model may still be undertrained or overfitting to certain scenarios. These unclear patterns highlighted the need for:

- Additional training epochs
- Improved observation preprocessing
- Possibly deeper model architectures for more stable decision boundaries



This experiment was instrumental in helping me understand the practical challenges of explainability in DRL and shaped the direction for more advanced interpretability approaches, such as **LMUT**, in the later stages of the project.

4.2 Implementing Linear Model U-Trees (LMUT)

One of my major contributions to the PEARL framework was the **full implementation of the Linear Model U-Trees (LMUT) Explainability Module**. This method provides **interpretable explanations** for Deep Reinforcement Learning (DRL) agents by approximating their Q-functions using a combination of decision trees and linear models.

Unlike SHAP or LIME, which treat the agent as a black box and focus on local feature importance, LMUT offers a **structurally transparent, model-mimicking approach**. It approximates the Q-function using decision tree partitions, where each leaf node contains a linear model that captures local relationships between inputs and value estimates.

4.2.1 Core Idea

The motivation behind LMUT is to **reconstruct the agent's learned behavior** using a simpler, more interpretable surrogate model. By fitting a decision tree to partition the input space and training **linear regressors at each leaf node**, the method allows us to:

- Understand which regions of the observation space lead to which actions
- Extract meaningful rules and feature weights per region
- Visualize how decisions vary based on input variation

LMUT bridges the gap between **performance and explainability**, offering **rule-based interpretability with numerical attribution**.

4.2.2 Implementation Highlights

I designed and implemented LMUT as a complete subclass within our explainability method interface. The module integrates seamlessly with the PEARL pipeline and includes the following components:

4.2.2.3 Per-Leaf Linear Model Training

For every unique leaf node, I trained a corresponding **LinearRegression** model on the samples that fall into that region. This allows for **region-specific explanation** of Q-values.

```
for leaf in unique_leaves:
    if np.sum(mask) > 1: # Only fit if we have enough samples
        linear_model.fit(X[mask], y[mask, i])
```

4.2.2.4 Explanation Generation

During inference, the model:

- Identifies which tree node a new input falls into
- Uses the linear model from that leaf to generate **feature importance values**
- Combines the tree's feature splits and linear coefficients into a **single interpretable explanation map**

To visualize relevance, I reshaped the importance scores back into the original observation shape and expanded them across all actions.

4.2.2.5 Integration with Feature Masking

Finally, I extended the `value()` method to compute **relevance scores** using our internal feature masking interface. This allowed us to compare saliency across different XAI methods in a unified framework.

4.2.2.6 Sample Code Snippet – LMUT Training & Explanation

The following code snippet summarizes how LMUT collects data, fits its surrogate models, and generates explanations from trained reinforcement learning agents:

```
# Collect data from environment-agent interactions
lmut_explainer.collect_training_data(num_samples=10000)

# Train the decision tree and per-leaf linear models
lmut_explainer.fit_models()

# Generate explanation map for a new observation
obs = env.get_observations()
importance_maps = lmut_explainer.explain(obs)

# Get relevance score for agent's actual action
scores = lmut_explainer.value(obs)
```

Each step is modular and compatible with different types of observations and agents. The result is an interpretable feature importance map per agent, which is visualized or evaluated downstream in the PEARL pipeline.

4.2.3 Practical Value

LMUT provides **localized, understandable feedback** on how an agent makes decisions, and **why**. It allows us to:

- Quantify feature importance per region and per agent
- Analyze consistency across partitions
- Detect spurious correlations or overfitting tendencies
- Support model debugging through interpretable visualization

It was especially valuable in our **Ericsson-sponsored use case**, where decision transparency was a key business requirement.

4.3 Integration of the Linear Model U-Trees (LMUT) Method into the Library and GUI

As part of our efforts to unify all explainability methods within the PEARL framework, we designed an initial integration plan for the LMUT (Linear Model U-Trees) method into the user interface (UI) layer. A dedicated screen was created to visualize LMUT explanations, consistent with the visual language used across other explainability techniques like SHAP, LIME, and Policy Stability.

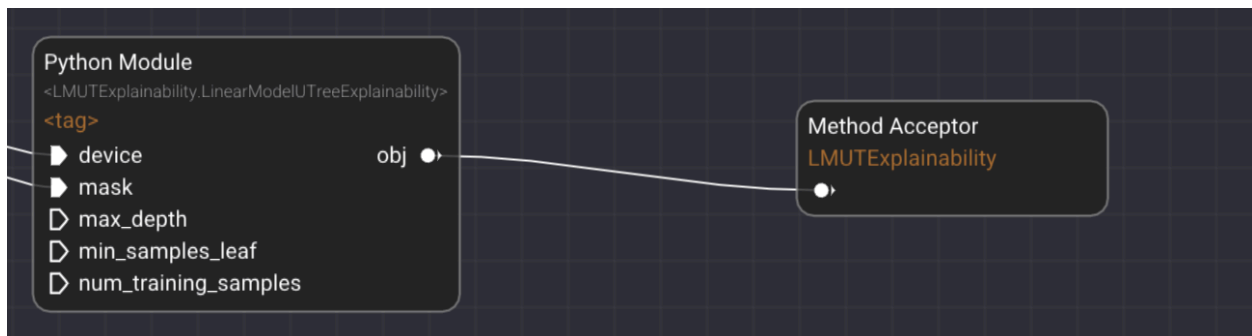
However, the integration of LMUT presented unique challenges that distinguished it from other methods. Unlike black-box approaches, LMUT operates as a **mimic learning technique**, requiring a full **data collection phase** and **model training step** before explanations can be generated. This process involves:

- Running pre-trained RL agents through the environment
- Capturing a sufficient number of observation–Q-value pairs
- Fitting decision trees and per-leaf linear models

Due to this additional training cycle and data dependency, LMUT was found to **interrupt the real-time, on-demand flow** of our existing explainability interface, which prioritizes instant feedback and live interaction.

For these reasons, we made a conscious decision to defer full UI integration of LMUT to future iterations of the PEARL project. While a placeholder screen was implemented in the GUI to accommodate LMUT explanations, we disabled this feature in the production release to preserve the performance and usability consistency of the interface.

We plan to revisit LMUT integration in the future with a background training pipeline or offline visualization mode that better aligns with the architectural flow of the application.

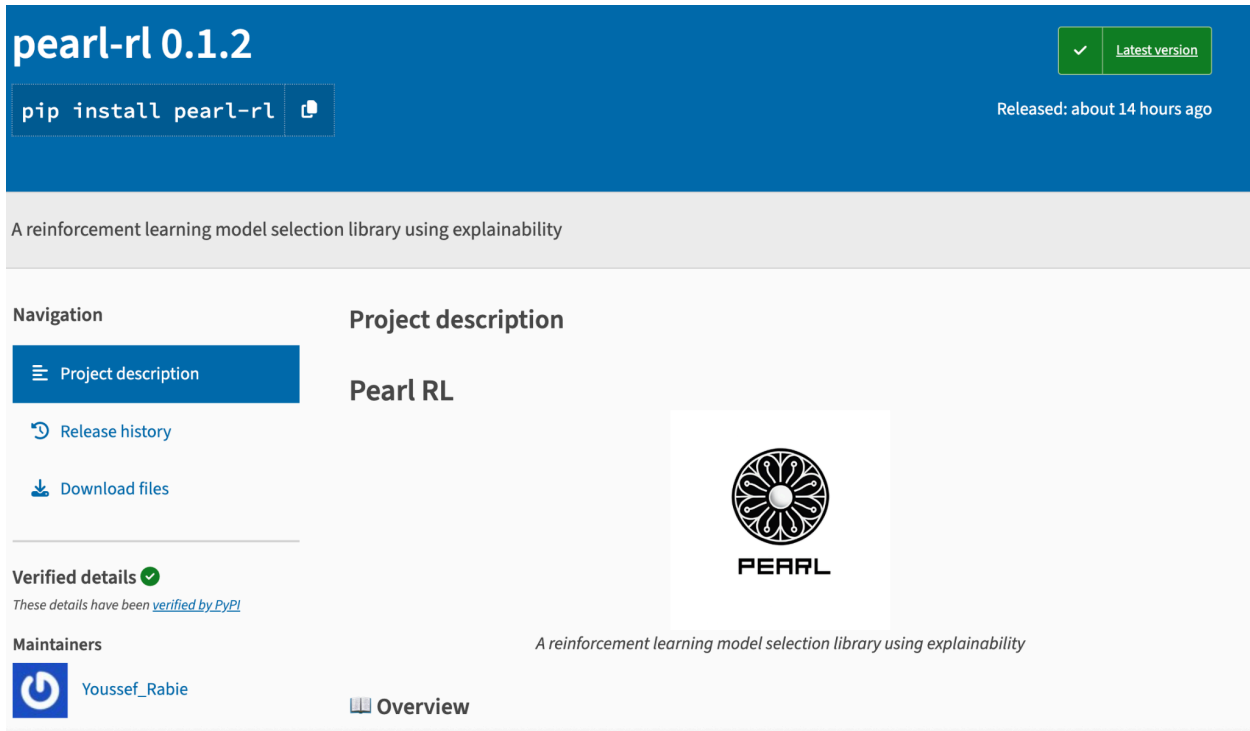


4.4 Deployment of the PEARL Library to PyPI

4.4.1 Overview

To make the PEARL framework easily accessible and installable by the wider machine learning community, we packaged and published it as a Python module under the name: **pearl-rl**

Available at: <https://pypi.org/project/pearl-rl/>

The screenshot shows the PyPI page for the 'pearl-rl' package. The header is blue with the package name 'pearl-rl 0.1.2' in white. A green button with a checkmark and 'Latest version' is in the top right. Below the header, a dark blue bar contains the command 'pip install pearl-rl' and a release status 'Released: about 14 hours ago'. The main content area has a light gray background. On the left, a 'Navigation' sidebar lists 'Project description' (selected), 'Release history', and 'Download files'. Below this, it says 'Verified details' with a green checkmark and a link to 'verified by PyPI'. The 'Maintainers' section shows a profile for 'Youssef_Rabie'. The main content area is titled 'Project description' and 'Pearl RL'. It features a circular logo with the word 'PEARL' below it. The description text reads 'A reinforcement learning model selection library using explainability'. At the bottom right, there is an 'Overview' link.

4.4.2 Installation

The library can be installed using the standard Python package manager:

```
yousefrabia@Yousefs-MacBook-Pro Pearl % pip install pearl-rl
```

4.4.3 Packaging and Build Process

To automate the process of building and uploading the package to PyPI, I developed a custom deployment script in Python that performs the following tasks:

- Cleans previous build artifacts
- Builds the package using build
- Checks the validity of the distribution using twine
- Uploads the package to either TestPyPI or the official PyPI based on user selection

4.4.4 Deployment Script Snippet:

```
run_command("pip install --upgrade build twine")
clean_build()
build_package()
check_package()
upload_to_pypi()
```

This script ensures that the package is clean, valid, and ready for public distribution.

4.4.5 Testing the Package

Before uploading, I also implemented a test script to validate the functionality of the package. The script performs:

- **Import tests:** Ensures all core modules and classes are importable
- **Version check:** Confirms version info is embedded correctly
- **Functional tests:** Mocks a training cycle using a dummy Agent and Environment

4.4.6 Summary

The publishing of pearl-rl to PyPI allows other developers, students, and researchers to easily integrate, test, and extend our explainability framework. This step marks the transition of PEARL from a research project to a reusable open-source tool, and reinforces our commitment to reproducibility and community contribution.

References

1. Terra, A., Inam, R., & Fersman, E. (2022).
BEERL: Both Ends Explanations for Reinforcement Learning. Applied Sciences,
DOI: [10.3390/app122110947](https://doi.org/10.3390/app122110947)
2. Liu, G., Schulte, O., Zhu, W., & Li, Q. (2018).
Toward Interpretable Deep Reinforcement Learning with Linear Model U-Trees.
arXiv preprint: [arXiv:1807.05887](https://arxiv.org/abs/1807.05887)
3. Sharma, V.
Explaining Deep Reinforcement Learning Models with Linear Model U-Trees.
Medium article, available at:
<https://vishnudsharma.medium.com/explaining-deep-reinforcement-learning-models-with-linear-model-u-trees-64c7d788fe4f>
4. Renott, N.
Deep Reinforcement Learning for Atari Games Python Tutorial | AI Plays Space Invaders.
YouTube Video, available at:
<https://www.youtube.com/watch?v=hCeJeq8U0Io>
5. Silver, D.
DeepMind x UCL | Reinforcement Learning Course.
University College London, YouTube Playlist:
<https://www.youtube.com/playlist?list=PLqYmG7hTraZDM-OYHWgPebj2MfCFzFObQ>
6. freeCodeCamp.org.
Reinforcement Learning Course – Full Machine Learning Tutorial.
YouTube Video, available at:
https://www.youtube.com/watch?v=ELE2_Mftqoc
7. DeepFindr.
Explainable AI – YouTube Playlist.
Available at:
<https://www.youtube.com/playlist?list=PLV8yxwGOxvvovp-j6ztxhF3QcKXT6vORU>